Advanced Database Systems
Final Portfolio Summary

Name: Ezekiel Oladoja | **Student ID:** 301931918 | **Course:** MIT 8103 | **Session:** 2026/2027 First Semester

Case Study: Hospital / Clinic Management System

GitHub Repository: https://github.com/eoladoja31918/Oladoja_Ezekiel_301931918/

Project scope and case study

This portfolio applies advanced database concepts to a simulated clinic providing consultations, diagnosis, prescriptions, and laboratory services. Receptionists, doctors, nurses, laboratory staff, and administrators require patient registration, appointment booking, diagnosis recording, test requests, report retrieval, and patient history. All sample records are fictional. PostgreSQL on Neon supports structured clinical relationships; MongoDB Atlas demonstrates a document model for laboratory reports. The two systems are separate demonstrations, with no automatic synchronization.

Portfolio 1 Database design and modelling

The relational model contains ten tables: patients, doctors, appointments, diagnoses, prescriptions, prescription_items, laboratory_requests, laboratory_tests, laboratory_request_items, and laboratory_results. The ER diagram identifies keys and cardinalities. Generated primary keys, unique business identifiers, foreign keys, required fields, and value checks protect integrity. Exclusion constraints prevent overlapping scheduled appointments for the same doctor or patient. Deferred triggers require medication items for finalised prescriptions and requested tests for submitted laboratory requests.

Normalisation progresses from repeating medication groups to individual item rows in 1NF, separates header facts from item facts in 2NF, and removes patient, doctor, and catalogue details from dependent records in 3NF. This reduces duplication and insertion, update, and deletion anomalies. Deliverables include the case study, business rules, ER diagram, schema, sample data, and normalisation explanation. A 12-check validation script was supplied; its complete PASS grid has not been independently reviewed in the evidence available for this summary.

Portfolio 2 Query processing and optimisation

A synthetic batch adds 73,025 records, including 5,000 patients and 30,000 appointments. Four queries retrieve patient history, doctor schedules, pending tests, and monthly summaries. Actual Neon exports contain three baseline and three optimised executions per query. Partial indexes target scheduled appointments and pending laboratory work; Q3 also limits the ordered queue before descriptive joins. Existing integrity indexes were retained.

Query	Baseline ms	Optimised ms	Interpretation
Q1 Patient history	0.1050	0.1125	Essentially unchanged
Q2 Doctor schedule	0.8875	0.6580	25.86% reduction
Q3 Pending tests	6.6880	0.3205	95.21% reduction
Q4 Monthly summary	16.1205	16.0285	Essentially unchanged

Values are mean EXPLAIN Execution Time for runs 2 and 3; first runs are excluded to reduce cache-warming distortion. These small-sample results exclude planning and network time. Q3 full-row equivalence passed using bidirectional EXCEPT ALL. Q4 retained an appropriate full-table scan. Index storage and write overhead were not measured.

Portfolio 3 Transactions and concurrency

Transaction scripts create reserved P3 fixtures, commit an appointment and associated synthetic note, and roll back a second booking. The verification screenshot confirms one persisted booking/note and zero records for the discarded booking. This establishes the final state; it does not independently capture the second row before rollback.

The isolation exercise specifies READ COMMITTED: B should not see A's uncommitted doctor-name update, but may see the new value after A commits. The booking exercise uses overlapping appointments for different patients and the same doctor. One booking was inserted and committed, and a competing run showed an error. Final verification displays appointment 30015 for P3-P-01 with P3-D-01 on 1 February 2035, 09:00–09:30 WAT, status scheduled.

Evidence limits remain: isolation screenshots do not show the before/after name values, and both booking captures display PID 753. The scripts and evidence review document the intended order and the checks.

Portfolio 4 NoSQL and advanced data models

MongoDB collection clinic_nosql.laboratory_reports stores one document per requested laboratory test, embedding its measurements and a test-name snapshot. Patient identifiers are referenced rather than copying personal details. Numeric and qualitative values share a validated document structure. Appointment and request-item references in the seed documents are explicitly simulated and were not checked against PostgreSQL.

The setup screenshot confirms strict/error validation and four indexes: the default identifier index, unique report number, unique requested-test reference, and compound patient-history index. Five seed reports demonstrate completed, pending, and in-progress states. Queries retrieve patient history, filter nested test fields, use $elemMatch for measurements, and complete report P4-LAB-0002 through a single-document update. The later deployment inspection confirms that report is completed and the batch totals are three completed, one pending, and one in-progress.

Embedding suits bounded measurements that are normally retrieved together, while snapshots preserve historical test descriptions. Trade-offs include duplicated catalogue text and external-reference management. The explain screenshot shows two index keys examined for patient history; no MongoDB before/after performance gain is claimed. The initial update acknowledgement and before/after documents are not visible in the submitted screenshots.

Portfolio 5 Distributed and cloud databases

A read-only Atlas inspection captured on 21 October 2026 at approximately 22:19 WAT confirms replica set atlas-2t8ax2-shard-0, MongoDB 8.0.34, and three members: one PRIMARY and two SECONDARY, each reporting health 1. It confirms one collection, five reports, four indexes, and an acknowledged majority read of completed P4-LAB-0002. Ten inspection sections returned observations; default-concern inspection returned code 59, CommandNotFound.

The architecture discusses replication, election-related availability limits, primary reads, majority concerns, and a proposed patient-based hashed shard key. Existing global unique indexes must be redesigned before that shard key could be implemented. Failover, outage recovery, and sharding were not tested. Provider, region, and tier still require a reviewed Atlas screenshot; a healthy snapshot is not an uptime guarantee.

Evidence and repository organisation

The private repository is organised into five portfolio folders, each containing a README, scripts, design or analysis documents, and evidence. The assessor must be granted access before LMS submission. Repository URL, link resolution, collaborator acceptance, and progressive commit history must be checked directly in GitHub; they have not been verified through this summary.

Portfolio	Key evidence location relative to its folder
1 Design	diagrams/er-diagram.png; sql/schema.sql; normalisation.md
2 Optimisation	evidence/baseline-plans/baseline-plans.json; evidence/optimised-plans/optimised-baseline-plans.json
3 Transactions	evidence/commit-rollback-verification.png; evidence/verify.png; session screenshots
4 NoSQL	evidence/01-setup.png; evidence/02-seed.png; evidence/03-queries.png
5 Cloud	evidence/deployment-inspection.json; architecture.md

My Technical Reflection

This project helped me connect database theory with practical clinic workflows. Separating patients, doctors, appointments, and item records clarified how normalisation prevents repeated facts and update anomalies. The query work showed me to inspect actual plans rather than assume that every sequential scan is inefficient. Q3's measured improvement came from targeted indexes and reducing intermediate work; Q4 needed no forced index change.

Repeated inserts and transaction errors also showed why committed work must be inspected before rerunning scripts. The session screenshots taught me that executing two files does not prove concurrent execution: distinct persistent connections and ordered observations are necessary. MongoDB demonstrated how embedded measurements support varied reports, while validation and unique indexes still provide structure. Atlas inspection distinguished a healthy replica-set snapshot from tested failover. I would improve the project by completing the missing session evidence, validating cross-database references, measuring index costs, and testing recovery separately. These limits keep the conclusions aligned with what was actually observed.

References

1. MIT 8103 Advanced Database Systems. First Semester Continuous Assessment, September Cohort 2026, 2026/2027 academic session. Assessment brief supplied with this project.
2. PostgreSQL Global Development Group. PostgreSQL Documentation: constraints, transactions, isolation, and EXPLAIN. https://www.postgresql.org/docs/
3. MongoDB. MongoDB Manual: schema validation, indexes, replication, read concern, and sharding. https://www.mongodb.com/docs/manual/
4. MongoDB. Atlas Documentation: deployment and supported commands. https://www.mongodb.com/docs/atlas/

AI use declaration

I used ChatGPT to assist with planning, drafting SQL and mongosh scripts, interpreting supplied execution plans and screenshots, and preparing documentation. Verification included execution in Neon and Atlas, inspection of actual outputs, correction of the decimal Actual Rows conversion in plan capture, comparison of measured plans, and review of the report-state and topology evidence. AI-generated material remains my responsibility to understand and explain. Outcomes lacking sufficient evidence are identified above; they are not reported as completed tests.